

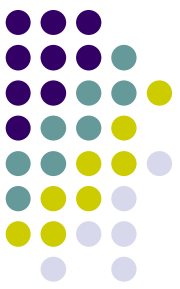
Visitor Pattern

Architecture of Computer Games
SE 456

Ed Keenan

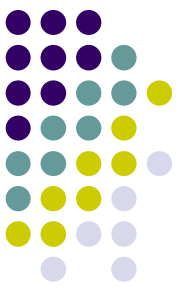
10 February 2021

13.0.8.4.13 Mayan Long Count



Collisions

- Big problem
 - Very quick efficient way to discriminate collision decisions?
 - Who collided with whom?
- Requirements
 - No conditionals
 - No switch statements
 - Extensible
 - Scalable

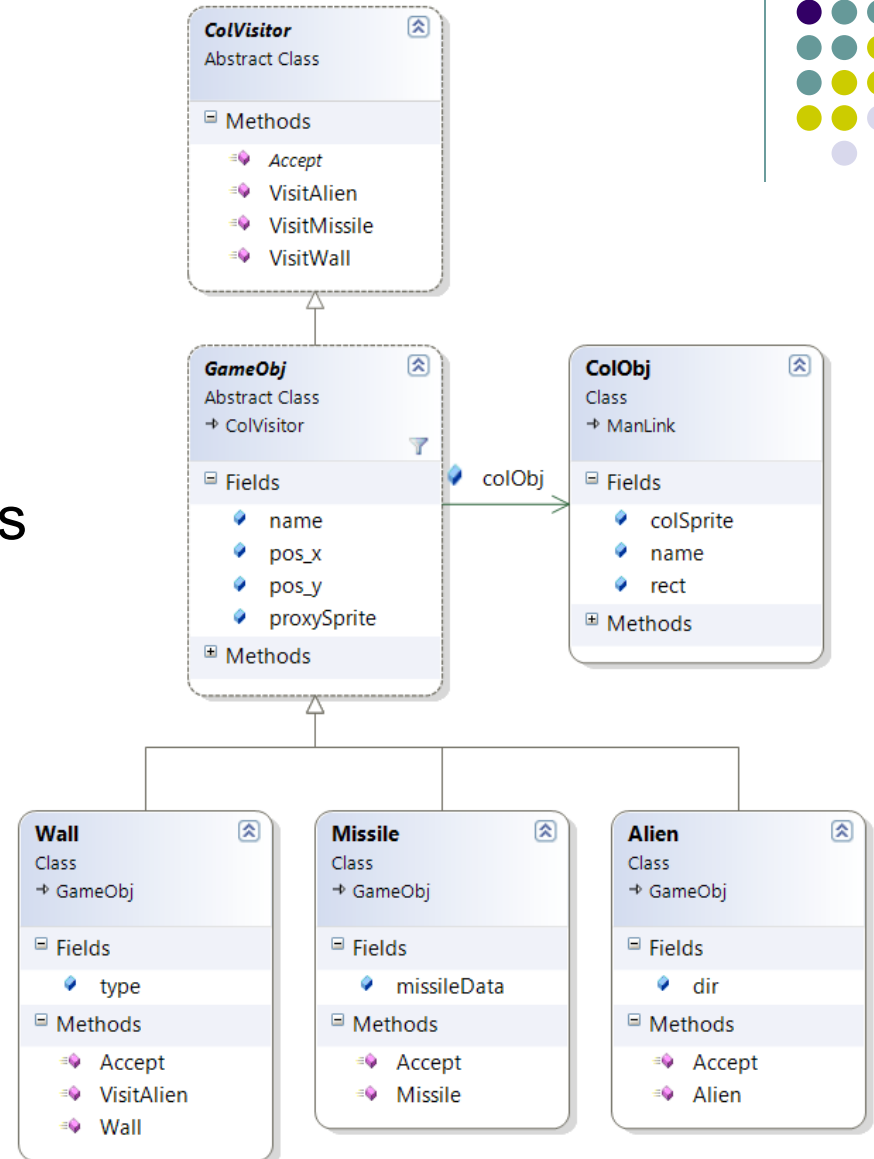


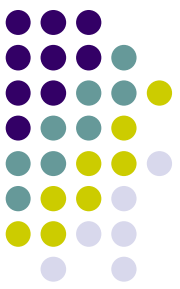
Who collided?

- Visitor pattern
 - This pattern is really being used backwards
 - Only the double dispatch portion
- Each GameObject
 - Has **Accept()** method
 - Has sparsely populated **VisitedBy()** methods for each accepted collision type.
- My Opinion
 - Clean solution to a sticky problem

Who collided?

- Pitfall
 - Colliding at a too low-level
 - Need to collide at GameObj level
- Only have Concrete Objects
 - Specialized GameObjs
 - Wall
 - Missile
 - Alien
- **Visitor Pattern**
 - Best for resolving which object collides

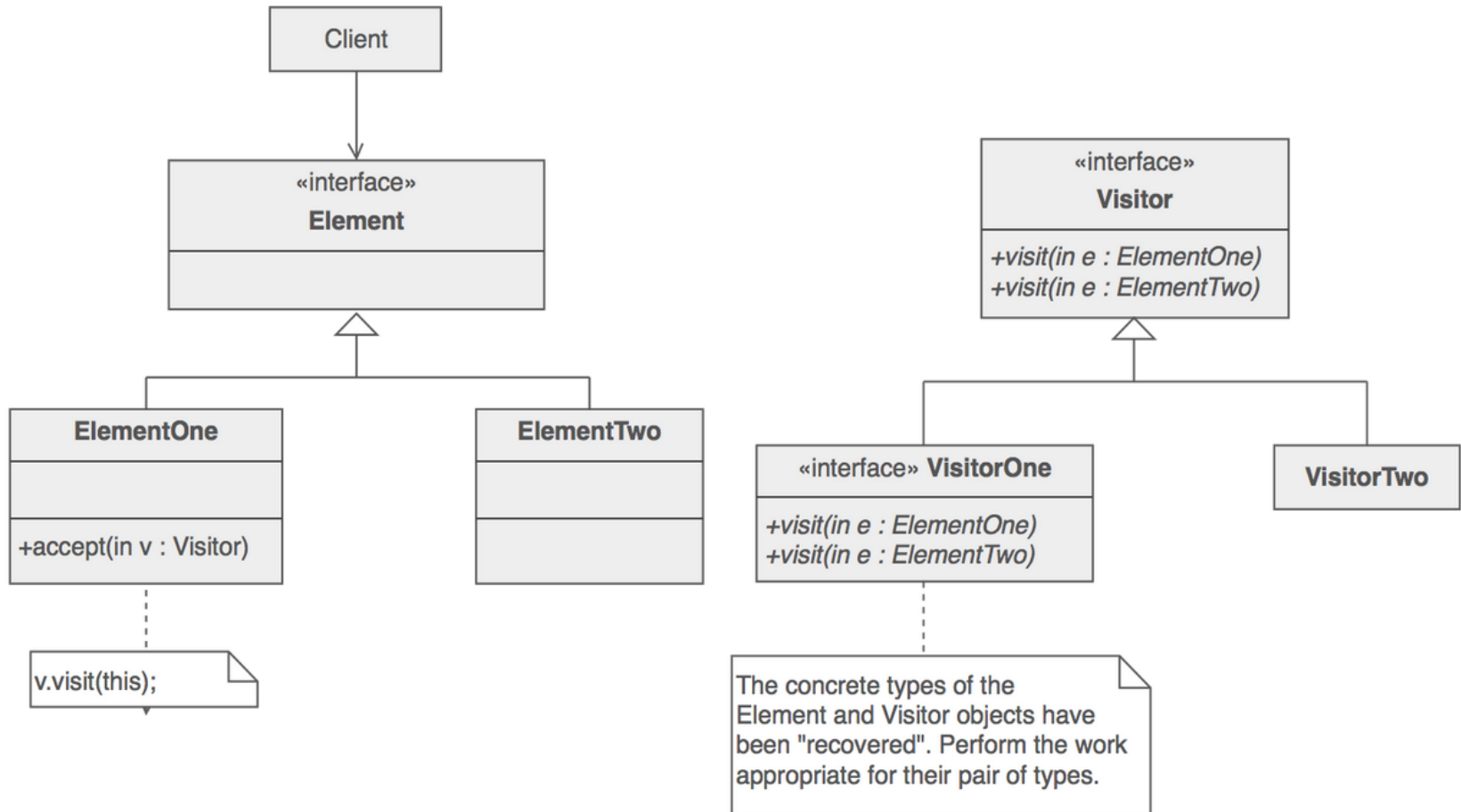
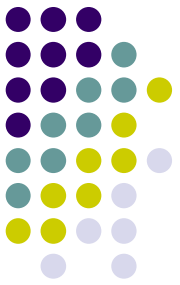


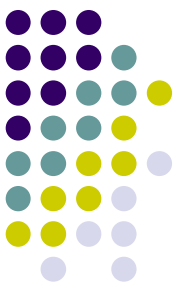


Visitor

- Intent
 - Represent an operation to be performed on the elements of an object structure.
 - Visitor lets you define a new operation without changing the classes of the elements on which it operates.
- Remarks
 - The classic technique for recovering lost type information.
 - Do the right thing based on the type of two objects.
 - Double dispatch

Visitor



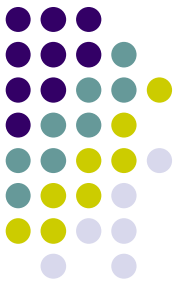


Design Patterns

- Easy tutorial book on design patterns
 - Get your copy today
 - Ecopy at Depaul Library through the safari link



Thank You!



- Questions?

